



UNIVERSIDAD DE MONTERREY

Integración de aplicaciones computacionales

Proyecto 4. Plataforma de Bienestar Laboral y Prevención de Burnout (EQUIPO 03) Monorepo y ambientes de ejecución

Integrantes:

Emiliano Pascual Piñales Sánchez #657657

Gerardo Quiroga Ramírez #650313

Ricardo Arath Sánchez Aguirre #583928

Juan Pablo Ramos Salazar #610248

“Damos nuestra palabra que hemos realizado esta actividad con integridad académica”

Contenido

Para actualizar el índice en Word: clic derecho sobre la tabla → Actualizar campos → Actualizar toda la tabla.

1. Propósito

Este documento describe el repositorio del proyecto y los tres ambientes de ejecución que exige la rúbrica: developer, staging y production. Explica cómo está organizado el código, cómo se levanta cada ambiente, qué cambia entre uno y otro, y qué comprobaciones impiden que un error de operación pase desapercibido.

Todo lo que se afirma aquí fue ejecutado. La sección 8 contiene la evidencia: la cadena de arranque de la base corrida de principio a fin, la prueba de humo de cinco pasos y las pruebas de frontera de privilegios, con la salida textual de cada una.

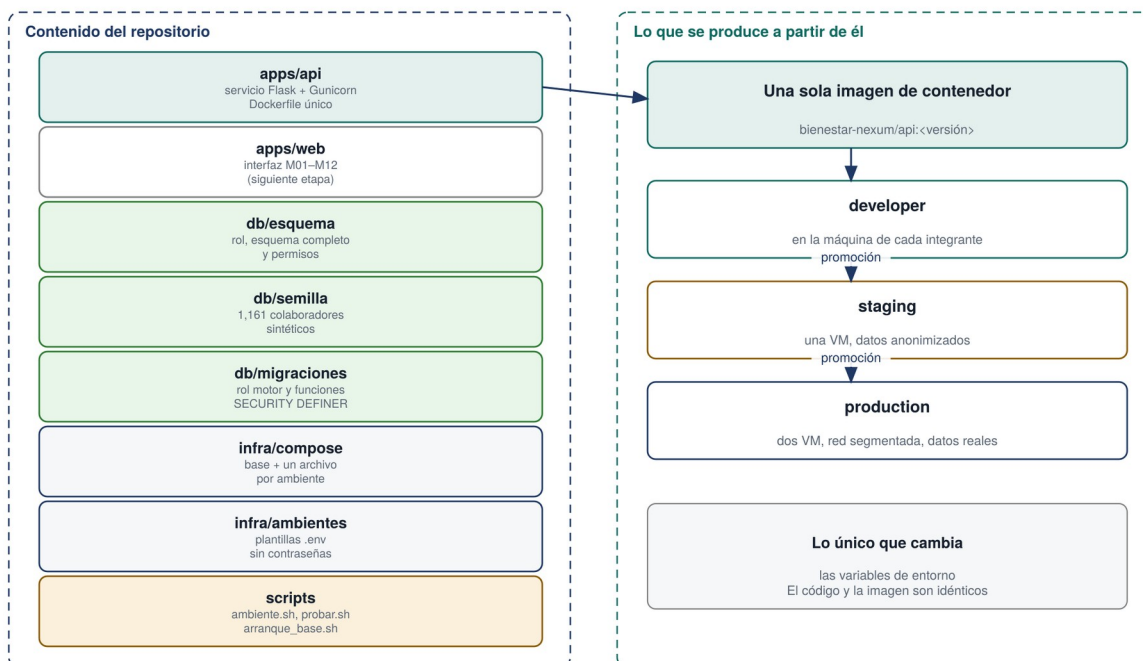
2. Por qué un monorepo

El proyecto tiene cuatro piezas que cambian juntas: el esquema de la base de datos, el servicio de aplicación, la interfaz y la definición de infraestructura. Separarlas en cuatro repositorios obligaría a coordinar versiones entre ellos a mano.

En un monorepo, un cambio de esquema y el cambio de código que lo necesita entran en el mismo commit. No pueden desincronizarse, porque no existe una versión del repositorio en la que uno esté y el otro no. Para un equipo de cuatro personas trabajando en paralelo durante un semestre, esa propiedad vale más que la independencia de despliegue que daría la separación.

El costo aceptado es que la tubería de integración continua corre sobre todo el repositorio aunque el cambio toque una sola carpeta. Con el tamaño actual del proyecto, ese costo es de minutos.

Estructura del monorepo y qué alimenta cada carpeta



El repositorio es uno solo: base de datos, aplicación e infraestructura viajan juntas y versionadas.

Un cambio de esquema y el cambio de código que lo necesita entran en el mismo commit; no pueden desincronizarse.

Figura 1. Estructura del monorepo y lo que se produce a partir de él.

3. Organización del repositorio

Ruta	Contenido y razón de ser
apps/api	Servicio de aplicación en Flask servido por Gunicorn. Contiene el Dockerfile único que alimenta los tres ambientes.
apps/web	Interfaz de los módulos M01 a M12. Se construye en la siguiente etapa sobre la base ya verificada.
db/esquema	Cadena de arranque de la base: creación del rol de aplicación, esquema completo con el mecanismo de privacidad diferencial, y otorgamiento de permisos de menor privilegio.
db/semilla	Generador de volumen sintético: 1,161 colaboradores en 16 unidades, cuatro campañas, 19,020 reactivos respondidos.
db/migraciones	Cambios de estructura posteriores al esquema base, numerados y versionados.
infra/compose	Un archivo base con lo común y un archivo por ambiente con lo que cambia. Nunca se usa el base solo.
infra/ambientes	Plantillas de variables de entorno. Los archivos con valores reales están excluidos del control de versiones.
infra/nginx	Configuración del proxy de staging y production, incluidos los encabezados de seguridad.
infra/secretos	Vacío en el repositorio por diseño. Los secretos se inyectan al desplegar.
scripts	Operación de ambientes, prueba de humo y carga de la base sin Docker.
docs	Documentación del proyecto.

3.1 Qué no está en el repositorio

Tres cosas quedan fuera a propósito, y el archivo .gitignore lo hace cumplir:

- Los archivos de variables con valores reales. Solo se versionan las plantillas terminadas en .ejemplo.
- Los secretos y los certificados. La carpeta infra/secretos contiene únicamente un archivo de instrucciones.
- Cualquier volcado de datos de la organización. El único volumen de datos versionado es el generador sintético.

Consecuencia práctica: si el repositorio se hiciera público por accidente, no habría ninguna credencial que rotar ni ningún dato personal expuesto. Es una decisión barata de tomar al inicio y muy cara de corregir después.

4. Los tres ambientes

Los tres ambientes ejecutan la misma imagen de contenedor. Lo único que cambia son las variables de entorno que se le inyectan. Esa es la propiedad que permite afirmar que lo probado en staging es lo que corre en production: no es una afirmación de proceso, es una consecuencia de que el artefacto sea el mismo.

Aspecto	developer	staging	production
Dónde corre	Máquina de cada integrante	Una VM e2-small	VM e2-small + VM e2-medium
Datos	Sintéticos por script	Copia anonimizada	Reales de la organización
Puerto de PostgreSQL	Publicado (5432)	No publicado	No publicado

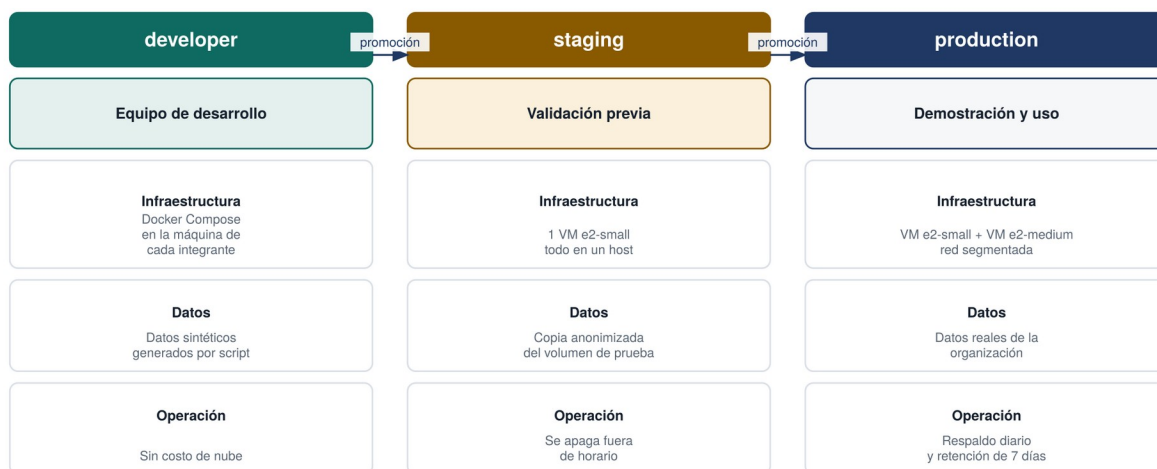
Aspecto	developer	staging	production
Procesos de Unicorn	1, con recarga	1	4
Detalle de error al cliente	Sí	No	No
Versión de la imagen	local	Etiqueta	Obligatoria
Respaldo	No	No	Diario, 7 días
Costo de nube	Ninguno	Se apaga fuera de horario	Continuo

4.1 Por qué existe staging

Staging no es una copia decorativa de production. Existe para que la demostración no sea la primera vez que el sistema corre fuera de una laptop. Los problemas que aparecen al salir de la máquina de desarrollo —resolución de nombres entre contenedores, permisos de archivos, variables que faltan, tiempos de arranque— se descubren ahí, con datos anonimizados y sin consecuencias.

Por eso staging también tiene proxy, aunque sin certificado propio: lo que se prueba es el camino completo de la petición, no el certificado.

Los tres ambientes exigidos: developer, staging y production



El mismo Dockerfile y las mismas imágenes recorren los tres ambientes. Lo único que cambia son las variables de entorno, que viven en Secret Manager y nunca en el repositorio. Un cambio se prueba en developer, se valida en staging y se promueve.

Staging existe para que la demostración en production no sea la primera vez que el sistema corre fuera de una laptop.

Figura 2. Comparación de los tres ambientes.

5. Cómo se levanta un ambiente

Un solo comando opera cualquiera de los tres. El script resuelve la combinación de archivos de compose y el archivo de variables correcto, de modo que nadie tenga que recordarla ni pueda equivocarse al escribirla.

```
# preparar variables la primera vez
cp infra/ambientes/developer.env.ejemplo infra/ambientes/developer.env
```

```
./scripts/ambiente.sh developer arriba # levanta los tres servicios
./scripts/ambiente.sh developer probar # prueba de humo de cinco pasos
./scripts/ambiente.sh developer bitacora # sigue la bitácora
./scripts/ambiente.sh developer abajo # apaga
./scripts/ambiente.sh developer borrar # apaga y elimina los datos
```

El primer arranque crea el volumen de datos y ejecuta la cadena completa: rol de aplicación, esquema, volumen sintético, migración y permisos. Arranques posteriores reutilizan el volumen y no vuelven a ejecutarla.

5.1 Orden de la cadena de arranque

El entrypoint de PostgreSQL ejecuta en orden alfabético los archivos de su carpeta de inicialización, y no entra a subdirectorios. Por eso los archivos se montan uno por uno con prefijo numérico, y no como carpeta:

Orden	Archivo	Qué hace
00	00_rol_aplicacion.sh	Crea el rol con el que se conecta la aplicación, sin privilegios todavía.
01	01_esquema.sql	Esquema completo, catálogos, funciones de privacidad diferencial y datos base.
02	02_volumen_sintetico.sql	Volumen de prueba. Solo en developer y staging; nunca en production.
05	001_rol_motor_...sql	Crea el rol motor y convierte las funciones de dominio en SECURITY DEFINER.
09	09_permisos.sh	Otorga a la aplicación exactamente lo que necesita, y nada más.

El mismo orden se puede ejecutar contra una instalación de PostgreSQL existente, sin Docker, con `scripts/arranque_base.sh`. Eso permite trabajar aunque no se puedan descargar imágenes de contenedor.

6. Frontera de privilegios

Esta sección describe el hallazgo de mayor peso de esta etapa, y conviene leerla completa porque cambió el diseño.

6.1 El problema encontrado

Al aplicar el principio de menor privilegio al rol de la aplicación, se le negó la lectura del esquema de autorreporte y la escritura del presupuesto de privacidad. Las tres pruebas de frontera pasaron: la aplicación ya no podía leer respuestas individuales, ni borrar la bitácora, ni alterar el presupuesto.

Pero la función de agregado protegido también dejó de funcionar:

```
ERROR: permission denied for schema autoreporte
LINE 1: ... COUNT(DISTINCT r.respuesta_id) FROM autoreport...
```

La causa: la función corría con los privilegios de quien la invocaba. Al quitarle a la aplicación el acceso a las respuestas, se lo quitamos también a la función que la aplicación llama.

6.2 La solución equivocada y la correcta

La **solución equivocada** sería devolverle a la aplicación el permiso de lectura sobre autorreporte. Funcionaría de inmediato, y destruiría la garantía central del proyecto: cualquier consulta de la aplicación

podría entonces leer respuestas individuales y saltarse el mecanismo de privacidad diferencial por completo.

La solución adoptada fue crear un rol motor sin inicio de sesión, dueño de las funciones de dominio, que sí puede leer respuestas y escribir el presupuesto. Las funciones se marcaron SECURITY DEFINER, de modo que corren con los privilegios del motor y no con los de quien las llama.

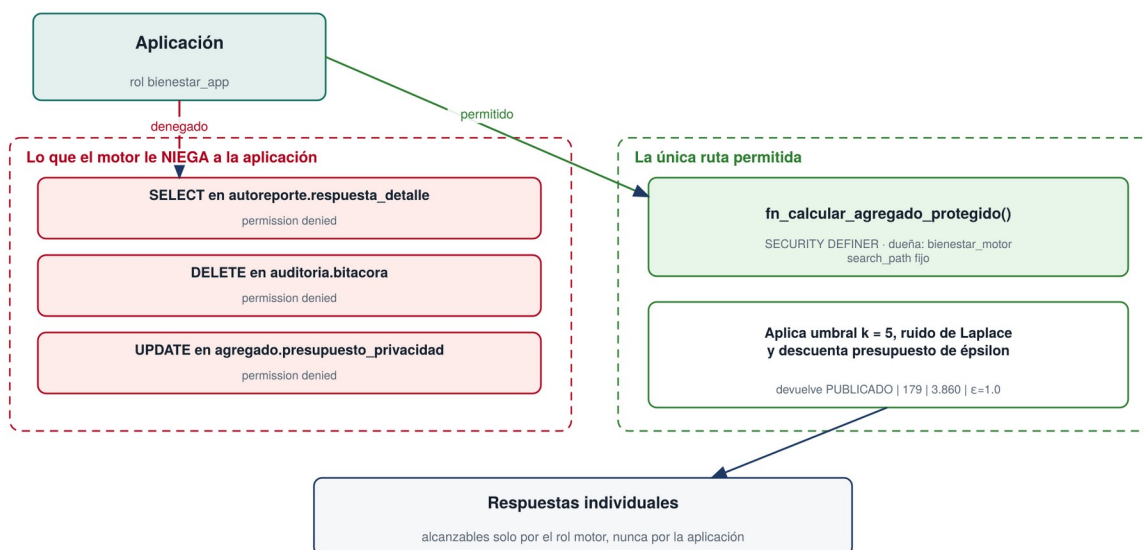
El resultado es que la única ruta desde la aplicación hacia los datos individuales pasa por el mecanismo de privacidad diferencial. No es una convención de programación que alguien pueda olvidar: es una restricción que el motor de base de datos hace cumplir.

6.3 La precaución que acompaña a SECURITY DEFINER

Toda función SECURITY DEFINER de este proyecto fija su search_path de forma explícita. Sin esa fijación, quien la invoca podría anteponer un esquema propio y hacer que la función ejecute tablas o funciones suyas con los privilegios del dueño. Es la vía clásica de escalación de privilegios en PostgreSQL, y la razón de las cláusulas SET search_path de la migración 001.

Por la misma razón, las funciones auxiliares que solo hacen cálculo —el ruido de Laplace, el análisis de utilidad y la clasificación de precisión— se dejaron deliberadamente con los privilegios de quien las invoca. Cada función con privilegios prestados es superficie de ataque, y esas tres no los necesitan.

Frontera de privilegios: la única ruta hacia los datos individuales



La garantía no depende de que la aplicación esté bien programada: depende de un permiso que la aplicación no tiene.

Un error de programación, o incluso una inyección de SQL, no alcanza las respuestas individuales.

Figura 3. La única ruta permitida hacia los datos individuales.

7. Guardas de operación

Tres comprobaciones están puestas para que un error de operación falle de forma visible en lugar de pasar desapercibido. Las tres se verificaron ejecutándolas.

7.1 Production exige versión explícita

El script de operación rechaza levantar production si el archivo de variables no declara una versión de imagen. Desplegar una etiqueta móvil como latest o local en production está prohibido, porque haría imposible saber qué código está corriendo.

```
$ ./scripts/ambiente.sh production estado
ERROR: production requiere VERSION en infra/ambientes/production.env.
      Desplegar 'local' o 'latest' en production está prohibido.
$ echo $?
1
```

7.2 Production no expone la base de datos

El archivo de production no tiene sección de puertos para el servicio de datos. La ausencia es deliberada y está comentada como tal, para que nadie la agregue creyendo que fue un olvido. Además, la red privada está marcada como interna, de modo que los contenedores de datos no tienen ruta de salida a Internet.

Comprobación	developer	production
Puerto de PostgreSQL publicado	5432	ninguno
Puerto de Redis publicado	6379	ninguno
Red privada marcada interna	sí	sí
Procesos de Unicorn	1	4
Límite de memoria del contenedor de datos	sin límite	4 GB

7.3 La tubería de integración continua protege la garantía

La tubería definida en `.github/workflows/ci.yml` comprueba en cada cambio que el rol de la aplicación sigue sin poder leer respuestas individuales ni borrar la bitácora, y que las seis garantías de gobernanza se cumplen. Si alguien relaja un permiso por comodidad, la tubería falla.

Esa es la razón de ser de la tubería en este proyecto: no vigila el estilo del código, vigila la propiedad de privacidad que el proyecto promete.

8. Evidencia de ejecución

Todo lo de esta sección es salida textual de comandos ejecutados, no ejemplos redactados.

8.1 Cadena de arranque de la base

```
$ ./scripts/arranque_base.sh bienestar_limpio --con-semilla
==> 00 rol de aplicación
==> 01 esquema
==> 02 volumen sintético
==> 05 migración 001 (rol motor y funciones SECURITY DEFINER)
==> 09 permisos de menor privilegio
Base bienestar_limpio lista.
```

8.2 Pruebas de frontera de privilegios

Cuatro intentos desde el rol de la aplicación. Los tres primeros deben fallar; el cuarto debe funcionar. Es la combinación la que demuestra el diseño: negar sin la cuarta línea sería un sistema inútil, y permitir sin las tres primeras sería un sistema sin garantía.

```

A) SELECT count(*) FROM autoreporte.respuesta_detalle;
   ERROR: permission denied for schema autoreporte

B) DELETE FROM auditoria.bitacora;
   ERROR: permission denied for table bitacora

C) UPDATE agregado.presupuesto_privacidad SET epsilon_consumido=0;
   ERROR: permission denied for table presupuesto_privacidad

D) SELECT estado,n,valor_protegido,epsilon
   FROM agregado.fn_calcular_agregado_protegido(
        'CAMP-2026-S1','U0-OPS-CC-T1',NULL,'USR-00003');
   PUBLICADO | 179 | 3.860 | 1.0

```

8.3 Las seis garantías, verificadas por el rol restringido

La verificación de gobernanza se ejecutó conectada como la aplicación, no como superusuario. Eso importa: comprueba que el rol restringido puede auditarse a sí mismo.

Garantía	Cumple	Detalle
G1 Intervenciones sin persona objetivo	sí	Ninguna tabla de intervención admite un empleado o seudónimo objetivo.
G2 Autorreporte sin identidad	sí	Ninguna tabla de autorreporte contiene empleado_id: solo seudónimo.
G3 Sin puntaje individual publicable	sí	Riesgo y agregación operan solo sobre unidades.
G4 Toda publicación lleva épsilon	sí	Ningún resultado publicado carece del mecanismo aplicado.
G5 Toda publicación respeta el umbral k	sí	Ningún resultado publicado proviene de un grupo menor que k.
G6 Fuentes de vigilancia bloqueadas	sí	Ningún instrumento se alimenta de una fuente no autorizada.

8.4 Prueba de humo del ambiente

```

$ ./scripts/probar.sh developer
Prueba de humo – ambiente developer
OK   el proceso responde
OK   el ambiente reportado es developer
OK   PostgreSQL y Redis contestan
OK   el agregado protegido se calcula
OK   la segunda consulta llega desde la caché
Resultado: 5/5

```

8.5 Sonatas de salud

El servicio separa la sonda de proceso vivo de la sonda de servicio listo. Un proceso vivo cuya base de datos no responde no debe recibir tráfico, y el balanceador necesita poder distinguir ambos casos.

```

$ curl -s localhost:8000/salud
{"ambiente":"developer","estado":"vivo"}

$ curl -s localhost:8000/listo
{"ambiente":"developer","estado":"listo","dependencias":{
  "postgresql":{"ok":true,"ms":14.3},
  "redis":{"ok":true,"ms":3.7}}}

```

8.6 Degradación con Redis caído

Se apagó Redis con la aplicación en marcha, para comprobar que la caché es una optimización y no una dependencia dura.

```
$ redis-cli shutdown nosave
$ curl -s -H 'X-Actor-Id: USR-00003' localhost:8000/agregado/CAMP-2026-S1/U0-CAL-T1
{"estado": "PUBLICADO", "n": 61, "valor_protegido": 3.864, "epsilon": 1.0,
 "escala_ruido": 0.065574,
 "mensaje": "Valor protegido con ruido de Laplace. Precision:
            PRECISION_SUFICIENTE (error esperado 0.07 puntos)."}
$ curl -s -o /dev/null -w '%{http_code}' localhost:8000/listo
503
```

Interpretación: la consulta siguió respondiendo correctamente desde la base de datos, y al mismo tiempo la sonda de servicio listo devolvió 503. Ese es el comportamiento deseado: el usuario no ve una falla, y el balanceador sabe que esa instancia está degradada y puede sacarla de rotación.

8.7 Consumo del presupuesto de privacidad a través de la aplicación

Tras las consultas de la prueba, el presupuesto quedó consumido en la cantidad exacta esperada. La aplicación no puede escribir esta tabla; el consumo lo hizo el mecanismo.

Unidad	Épsilon consumido	Total	Agotado
UO-OPS-CC-T1	1.00	8.0000	no
UO-CAL-T1	1.00	8.0000	no

9. Limitaciones declaradas

#	Limitación	Consecuencia y tratamiento
1	Las imágenes de contenedor no se pudieron descargar en el entorno donde se redactó este documento.	La configuración de los tres ambientes se verificó por resolución completa, y la cadena de arranque de la base y la aplicación se ejecutaron contra PostgreSQL y Redis reales. Falta ejecutar el arranque dentro de contenedores en una máquina del equipo.
2	El ambiente de production aún no está desplegado en Google Cloud Platform.	Definido en archivos y con las guardas puestas, pero sin ejecución real. Es requisito de la demostración y está en el sprint en curso.
3	La carpeta apps/web está vacía.	Los módulos M01 a M12 se construyen en la siguiente etapa, sobre la base ya verificada.
4	La tubería de integración continua no se ha ejecutado en GitHub Actions.	Sus comprobaciones se validaron localmente una por una; falta la primera corrida en el servicio.
5	No hay prueba de carga sobre los contenedores.	Las mediciones de rendimiento se hicieron sobre procesos nativos. Los contenedores añaden una capa de red que aún no se ha medido.

10. Conclusiones

- El repositorio reúne base de datos, aplicación e infraestructura en un solo árbol versionado, de modo que un cambio de esquema y el código que lo necesita no pueden desincronizarse.
- Los tres ambientes exigidos quedan definidos con una sola imagen de contenedor y tres conjuntos de variables. La paridad entre ambientes es una consecuencia del artefacto compartido, no una promesa de proceso.

- La aplicación de menor privilegio reveló un hueco real de diseño: la función de agregado corría con los privilegios de quien la llamaba. Se corrigió con un rol motor y funciones SECURITY DEFINER con search_path fijo.
- Como resultado, la única ruta desde la aplicación hacia los datos individuales pasa por el mecanismo de privacidad diferencial, y esa restricción la hace cumplir el motor de base de datos, no una convención de programación.
- Tres guardas de operación impiden los errores más caros: desplegar sin versión, exponer la base de datos y relajar un permiso sin que nadie se entere.
- Las seis garantías de gobernanza se cumplen verificadas desde el rol restringido de la aplicación, y la prueba de humo del ambiente pasa cinco de cinco.
- Lo que falta está declarado en la sección 9, con su tratamiento y su sprint.